Implementierung

Aufbau, Entscheidungen und Tests

Inhaltsverzeichnis

Aufbau 1
Wichtige Codeteile 2
    Neuron: Forward und Update 2
    Layer: Backpropagation 2
    Netz: ein Trainingsschritt 3
Entscheidungen 3
Fehler im ersten Entwurf 4
Tests 4
Selbst ausprobieren 5

Aufbau

Die Struktur folgt dem Vorschlag der Angabe: Ein **Netz** besteht aus **Layern**, ein Layer aus **Neuronen**. Python und C# haben dieselben Klassen und Methoden, nur jeweils in der Schreibweise der Sprache (`update_weights` bzw. `UpdateWeights`).

```
classDiagram
    class NeuralNetwork {
        layers : list~Layer~
        learning_rate : float
        predict(inputs) outputs
        train(inputs, targets) cost
        fit(inputs, targets, epochs) costs
        dump_json(path)
        load(path)
        dump_txt(path)
        load_txt(path)
        dump_initfile(path, prefix)
    }
    class Layer {
        neurons : list~Neuron~
        forward(inputs) outputs
        backward(errors, learning_rate) previous_errors
    }
    class Neuron {
        weights : array
        bias : float
        inputs : array
        output : float
```

```
        activate(inputs) output
        activate_derivative() float
        update_weights(delta, learning_rate)
        update_bias(delta, learning_rate)
    }
    NeuralNetwork "1" *-- "1..*" Layer
    Layer "1" *-- "1..*" Neuron
```

Klasse	Aufgabe
`Neuron`	speichert Gewichte und Bias, berechnet $a = \sigma(w \cdot x + b)$ und merkt sich Input und Output für die Backpropagation
`Layer`	ruft alle Neuronen auf; `backward` berechnet die Deltas, aktualisiert die Neuronen und gibt den Fehler an den vorherigen Layer zurück
`NeuralNetwork`	verkettet die Layer, trainiert (`train` = ein Beispiel, `fit` = ganze Epochen) und speichert bzw. lädt Netze

Erzeugt wird ein Netz mit einer Liste von Größen, deren erster Eintrag die Anzahl der Inputs ist:

```
# 2 inputs, 2 hidden neurons, 1 output; the seed makes the run reproducible
network = NeuralNetwork([2, 2, 1], learning_rate=0.1, seed=0)
network.fit(XOR_INPUTS, XOR_TARGETS, epochs=20000)
network.predict([1, 0])
```

Wichtige Codeteile

Neuron: Forward und Update

```
def activate(self, inputs: FloatArray) -> float:
    self.inputs = inputs
    z = float(np.dot(self.weights, inputs)) + self.bias
    self.output = self._activation_fn(z)
    return self.output

def update_weights(self, delta: float, learning_rate: float) -> None:
    # dJ/dw_i = dJ/dz * dz/dw_i = delta * x_i
    self.weights -= learning_rate * delta * self.inputs
```

Layer: Backpropagation

Das ist die Umsetzung von Delta und Gradient für einen Layer:

```python
def backward(self, errors: FloatArray, learning_rate: float) -> FloatArray:
    previous_errors = np.zeros(self.input_size)

    for neuron, error in zip(self.neurons, errors, strict=True):
        # dJ/dz = dJ/da * da/dz
        delta = error * neuron.activate_derivative()

        # dz/dx_i = w_i; must use the weights from the forward pass,
        # so this happens before the update below
        previous_errors += delta * neuron.weights

        neuron.update_weights(delta, learning_rate)
        neuron.update_bias(delta, learning_rate)

    return previous_errors
```

Netz: ein Trainingsschritt

```python
def train(self, inputs: ArrayLike, targets: ArrayLike) -> float:
    expected = np.asarray(targets, dtype=np.float64)
    outputs = self.predict(inputs)

    # dJ/dy_hat for J = 1/2 * (y_hat - y)^2
    errors = outputs - expected
    for layer in reversed(self.layers):
        errors = layer.backward(errors, self.learning_rate)

    return 0.5 * float(np.sum((outputs - expected) ** 2))
```

Entscheidungen

- **Der Input ist kein Layer** — so steht es in der Angabe ($X = A^{[0]}$). Ein eigener Input-Layer mit Gewichten würde aus dem 2-2-1-Netz ein Netz mit drei Gewichtsschichten machen.
- **Neuron-Objekte statt Matrizen** — die Struktur der Angabe ist leichter nachzuvollziehen. Eine Gewichtsmatrix pro Layer wäre deutlich schneller: Ein 7-Segment-Training über 20 000 Epochen dauert in Python rund 18 Sekunden.
- **Startgewichte gleichverteilt in $[-1, 1)$ mit festem Seed** — Neuronen starten in unterschiedliche Richtungen, und jeder Lauf ist wiederholbar.
- **Numerisch stabile Sigmoid-Funktion** — für negative z wird $\frac{e^z}{1+e^z}$ gerechnet, damit `exp` nie mit einer großen positiven Zahl aufgerufen wird und überläuft.
- **Eine Kommandozeile für alles** — `python -m neural_network xor`, `seven-segment` und `visualize` statt einzelner Skripte.
- **C# ohne Zusatzpakete** — single-threaded und mit einfachen Schleifen, damit man jeden Rechenschritt im Debugger verfolgen kann.

Fehler im ersten Entwurf

Unser erster C#-Entwurf hatte die richtige Struktur, aber mehrere Fehler, die das Training verhindert hätten. Sie sind in beiden Versionen behoben:

Stelle	Fehler	Richtig
`Neuron.Activate`	Sigmoid fehlte, die Ausgabe war nur z	$a = \sigma(z)$
`ActivateDerivative`	Sigmoid wurde ein zweites Mal angewendet	$a(1-a)$ mit der gespeicherten Ausgabe
`UpdateWeights` / `UpdateBias`	`w += gradient * w`, Lernrate unbenutzt	$w \mathrel{-}= \alpha\,\delta\,x$ bzw. $b \mathrel{-}= \alpha\,\delta$
`NeuralNetwork`	Input-Layer mit eigenen Gewichten; der Hidden-Layer bekam die Input-Größe statt der Neuronenzahl davor	Input ist $A^{[0]}$, jeder Layer bekommt die Größe des vorherigen
`Layer`	Felder für die Größen wurden nie gesetzt	Größen aus den Neuronen ableiten
Startgewichte	nur positiv in $[0, 1)$	$[-1, 1)$
Python-Gerüst	`np.random_uniform` gibt es nicht, `for i in neuronSize` ohne `range`, Tippfehler `active` statt `activate`	`rng.uniform`, `range(...)`, `activate` mit Rückgabewert

Tests

Die Python-Version hat 21 automatische Tests (`pytest`):

Test	Prüft
Gradienten-Check	Backpropagation gegen numerische Ableitung, drei Netzgrößen
Forward-Pass	Ergebnis gegen eine von Hand nachgerechnete Formel
Konvergenz	XOR lernt alle 4 Fälle, der 7-Segment-Decoder alle 10 Ziffern
Export/Import	JSON und TXT neu laden ergibt bitgleiche Vorhersagen; C-Header enthält jeden Wert
Fehlerfälle	falsche Layer-Größen, falsche Input-Länge, kaputte Dateien werden abgelehnt
Visualisierung	die erzeugte Seite enthält gültige Daten für alle Szenarien

Selbst ausprobieren

Im Ordner `python/` des Repos:

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m neural_network xor
.venv\Scripts\python   -m   neural_network   seven-segment   --export   ..
\models\python
.venv\Scripts\python -m neural_network visualize
.venv\Scripts\python -m pytest
```

Die C#-Version ist in Arbeit. Sie bekommt dieselben Befehle (`dotnet run -- xor`, `seven-segment`, `selftest`) und lädt die in Python trainierten Netze.